

Cztery poprawki wykryte przy testowaniu zatrzymania w trakcie ruchu. Wszystkie dotyczą niezawodności łącza serwer–sterownik.

Pliki

- `server/app/main.py` — jeden poller statusu w tle (`_poll_loop`) zamiast odpytywania z każdej pętli WebSocketu; przejście z `on_event` na `lifespan`.
- `server/app/machine.py` — odporność `_command()` na zerwane połączenie, komunikat alarmu ze sterownika.
- `bridge/sc4hub_bridge.cpp` — odpowiedź na STOP przerywający ruch, pole `MSG=` w `STATUS`.
- `firmware/clearcore/README.md` — opis pola `MSG=`.

Poprawione błędy

STOP w trakcie ruchu wieszał serwer. Linie STOP konsumował nasłuch w pętli ruchu, a przzerwana komenda ruchu zwracała pustą odpowiedź — więc na żądanie STOP nie szła żadna linia i serwer czekał do timeoutu. Teraz przzerwany ruch odpowiada OK zatrzymano, co odbiera właśnie żądanie STOP.

Serwer nie łączył się ponownie po restarcie sterownika. `_command()` łapało `OSError` i `TimeoutError`, ale zamknięty transport w uvloop rzuca `RuntimeError` — nieobsłużony, więc pętla statusu wywalala się w kółko i połączenie nigdy nie było odtwarzane. Doszło też sprawdzanie `is_closing()` przed zapisem i obsługa końca strumienia: pusta odpowiedź uchodziła wcześniej za poprawną.

Odpytywanie per WebSocket. Każde otwarte połączenie panelu odpytywało sterownik samodzielnie. Przy kilku panelach mnożyło to komendy, uchwyty rywalizowały o wspólny zamek, a uchwyt zablokowany w odpytywaniu nie zauważał rozłączenia klienta i zostawał na zawsze — nowy klient bywał zagłodzony. Teraz odpytuje jeden zadanie w tle, a WebSockets tylko wysyłają gotowy stan.

Efekt uboczny, wcześniej opisany jako ograniczenie: `/api/status` jest aktualne także bez otwartego panelu. Wcześniej bez WebSocketu stan zostawał na `INIT` i `START` odmawiał.

Alarm bez powodu. STATUS nie przynosił komunikatu, więc operator widział ALARM bez wyjaśnienia. Doszło pole `MSG=` — zawsze ostatnie w linii, bo tekst zawiera spację.

Zweryfikowane na sprzęcie

STOP w trakcie ruchu programu:

```
STOP -> HTTP 200 po 30 ms

ruch po zatrzymaniu: 0.000 mm (maszyna stoi)

stan=ALARM, wrzeczono wyłączone, alarm='zatrzymano przyciskiem STOP'

START przy alarmie -> 409 "start możliwy tylko w stanie READY (obecnie: ALARM)"

RESET -> NOT_HOMED, komunikat alarmu wyczyszczony
```

Nieprzetestowane

- **Utrata sygnału zezwolenia w trakcie ruchu** — wymaga fizycznego zadziałania układu bezpieczeństwa (wejście Global Stop huba). Ścieżka w kodzie jest gotowa: pętla oczekiwania na koniec ruchu sprawdza zezwolenie i przy jego utracie robi `NodeStop(STOP_TYPE_ABRUPT)`, wyłącza wrzeciono i przechodzi w `ALARM`.
- Mostek obsługuje **jednego klienta naraz**. Przy zawieszonym starym połączeniu nowy serwer nie zostanie obsłużony, dopóki tamto nie padnie.

Źródło: docs/zmiany/status-i-zatrzymanie.md — maszyna do ocinania wlewków płytek optyki